

LEXIMIND: A Multi-Task Encoder–Decoder Transformer for Summarization, Topic Classification, and Emotion Detection

Oliver Perrin
Department of Computer Science
Appalachian State University
perrinot@appstate.edu

January 2026

Abstract

We present LEXIMIND, a multi-task natural language processing system built on a custom reimplementation of the Transformer encoder–decoder architecture. Rather than treating a pre-trained model as a black box, we construct every component—attention mechanisms, feed-forward networks, normalization layers, and positional encodings—from first principles, then transfer pre-trained weights from Google’s FLAN-T5-base (272M parameters) through an explicit weight-mapping module. This hybrid approach provides full architectural transparency while retaining the linguistic knowledge captured during large-scale pre-training.

The system performs three tasks simultaneously through hard parameter sharing: **(1)** abstractive text summarization for literary and academic content, **(2)** 7-class topic classification, and **(3)** 28-label multi-label emotion detection. We train on a combined corpus of 95,898 samples—49K summarization pairs (arXiv abstracts and Project Gutenberg literary descriptions), 43K GoEmotions samples, and 3.4K topic samples—using a single shared encoder with task-specific heads.

Two architectural interventions prove critical for balanced multi-task learning: **learned attention pooling** for the emotion classification head (replacing standard mean pooling), and **temperature-based task sampling** during training. Together these eliminate negative transfer for emotion detection, improving sample-averaged F1 from 0.199 to 0.352 (+77%). The final model achieves a BERTScore F1 of 0.83 for summarization, 85.7% accuracy for topic classification (95% CI: [80.4%, 91.0%]), and a tuned macro F1 of 0.294 for emotion detection. A controlled comparison against BERT-base (110M parameters) shows that the encoder-only architecture outperforms on pure classification tasks, but cannot perform abstractive summarization—highlighting the cost and benefit of the encoder–decoder design. Training completes in approximately 9 hours on a single consumer GPU (NVIDIA RTX 4070, 12 GB).

All code, trained weights, and datasets are publicly available.¹

1 Introduction

The Transformer architecture [2] has become the dominant paradigm in natural language processing. Its core innovation—*self-attention*—allows every token in a sequence to attend to every other token in parallel, replacing the sequential processing of recurrent models with a fully parallelizable computation. Building on this foundation, encoder–decoder Transformers like T5 [3] cast *all* NLP tasks as text-to-text problems, and FLAN-T5 [4] further enhances T5 through instruction fine-tuning on over 1,000 diverse tasks.

Despite the power of these pre-trained models, they are typically consumed as opaque APIs—download, fine-tune, deploy. Their internal mechanics remain hidden behind framework abstractions,

¹Code: <https://github.com/OliverPerrin/LexiMind> Model: <https://huggingface.co/OliverPerrin/LexiMind-Model> Demo: <https://huggingface.co/spaces/OliverPerrin/LexiMind>

making it difficult to (a) understand *why* a model behaves as it does, (b) customize individual components for new tasks, or (c) diagnose training pathologies.

LEXIMIND takes a different approach. We implement a complete Transformer encoder–decoder from scratch, building each module—multi-head attention, gated feed-forward networks, RMSNorm, relative position bias—as a standalone, documented component. We then load pre-trained weights from FLAN-T5-base into this custom architecture through a carefully designed factory module that maps between parameter naming conventions. The result is a system that combines the linguistic knowledge of large-scale pre-training with the transparency and flexibility of a from-scratch implementation.

On top of this shared backbone, we attach three task-specific heads:

1. A **decoder with language modeling head** for abstractive summarization of literary and academic texts.
2. A **classification head with learned attention pooling** for 28-label multi-label emotion detection.
3. A **classification head with mean pooling** for 7-class topic classification.

The key contributions of this work are:

- A complete, modular Transformer reimplement compatible with T5-family weight loading, serving as both a practical system and an educational reference.
- A multi-task architecture supporting both generative (seq2seq) and discriminative (classification) tasks from a shared encoder.
- A demonstration that **architectural isolation of task-specific components (attention pooling) combined with balanced optimization (temperature-based task sampling) converts negative transfer to positive transfer** in heterogeneous multi-task learning—the central empirical finding of this work.
- A curated training corpus of 95,898 samples across literary, academic, and conversational domains.
- A complete training infrastructure with mixed-precision support, gradient checkpointing, `torch.compile`, and MLflow experiment tracking, designed to run on consumer GPUs.

This paper is organized as follows. Section 2 reviews the Transformer architecture and relevant prior work. Section 3 describes LEXIMIND’s architecture in detail, building from primitive components up to the full multi-task system. Section 4 explains the weight transfer mechanism from FLAN-T5. Section 6 covers the training pipeline and datasets. Section 7 presents experimental results, and Section 8 discusses findings and limitations.

2 Background and Related Work

2.1 The Transformer Architecture

The Transformer [2] consists of stacked *encoder* and *decoder* blocks. Each encoder block contains two sublayers: multi-head self-attention and a position-wise feed-forward network. Each decoder block adds a third sublayer: cross-attention over encoder outputs. Residual connections and layer normalization surround each sublayer.

Post-LN vs. Pre-LN. The original Transformer applies Layer Normalization [7] *after* each residual connection (Post-LN). Subsequent work [5] showed that applying normalization *before* each sublayer (Pre-LN) significantly improves training stability, especially for deep networks. T5 and most modern LLMs adopt Pre-LN, and so does LEXIMIND.

RMSNorm. Zhang and Sennrich [6] proposed Root Mean Square Layer Normalization (RMSNorm), which drops the mean-centering step of standard LayerNorm while retaining competitive performance. T5 uses RMSNorm for all normalization layers, with a learned scale parameter and no bias.

2.2 T5 and FLAN-T5

T5 [3] introduced a unified text-to-text framework: every NLP task is converted to a sequence-to-sequence problem by prepending a task-specific prefix (e.g., “summarize:”). T5 introduced several architectural choices that distinguish it from the original Transformer:

- *Relative position bias* instead of absolute sinusoidal embeddings—positions are encoded as a learned bias added to attention scores rather than to token embeddings.
- *No $\sqrt{d_k}$ scaling* in attention—unlike the original Transformer, T5 omits the $1/\sqrt{d_k}$ scaling factor from attention scores.
- *Gated feed-forward networks* with GELU activation—three projections ($W_{\text{gate}}, W_{\text{up}}, W_{\text{down}}$) instead of two.
- *Shared input/output embeddings*—the decoder’s output projection shares weights with the token embedding matrix.

FLAN-T5 [4] enhances T5 by instruction fine-tuning on 1,836 tasks, producing substantially better zero-shot and few-shot behavior. LEXIMIND loads pre-trained weights from FLAN-T5-base (272M parameters).

2.3 Multi-Task Learning

Multi-task learning (MTL) [8] trains a single model on multiple tasks simultaneously, encouraging shared representations and acting as an implicit regularizer. *Hard parameter sharing*—where lower layers are shared across tasks and only the final heads are task-specific—remains the dominant approach.

A known challenge in MTL is *negative transfer*: when jointly training on multiple tasks degrades performance on one or more tasks compared to single-task training. This can occur when tasks have conflicting gradient signals or when one task dominates the shared representation. We address this through attention pooling and temperature-based task sampling (Section 5).

3 Architecture

LEXIMIND implements a complete encoder–decoder Transformer from first principles, constructing the system bottom-up from five core modules: attention, feed-forward networks, normalization, positional encoding, and task-specific heads. Figure 1 shows the high-level system architecture.

3.1 Transformer Block Structure

Figure 2 illustrates the internal structure of encoder and decoder blocks. Both follow the Pre-LN pattern: normalization is applied *before* each sublayer, and the sublayer output is added back to its input via a residual connection.

The encoder block computes:

$$\mathbf{x}' = \mathbf{x} + \text{SelfAttn}(\text{RMSNorm}(\mathbf{x})) \quad (1)$$

$$\mathbf{x}'' = \mathbf{x}' + \text{FFN}(\text{RMSNorm}(\mathbf{x}')) \quad (2)$$

The decoder block adds cross-attention between the two sublayers:

$$\mathbf{y}' = \mathbf{y} + \text{MaskedSelfAttn}(\text{RMSNorm}(\mathbf{y})) \quad (3)$$

$$\mathbf{y}'' = \mathbf{y}' + \text{CrossAttn}(\text{RMSNorm}(\mathbf{y}'), \mathbf{h}_{\text{enc}}) \quad (4)$$

$$\mathbf{y}''' = \mathbf{y}'' + \text{FFN}(\text{RMSNorm}(\mathbf{y}'')) \quad (5)$$

where $\mathbf{h}_{\text{enc}}$ is the encoder output. Hidden states are clamped to prevent overflow during mixed-precision (bfloat16) training, following the HuggingFace T5 implementation.

3.2 Multi-Head Attention

Attention is the mechanism that allows the model to dynamically focus on different parts of the input when producing each output element. Given an input $\mathbf{X} \in \mathbb{R}^{n \times d}$, we project it into queries, keys, and values:

$$\mathbf{Q} = \mathbf{X}\mathbf{W}_Q, \quad \mathbf{K} = \mathbf{X}\mathbf{W}_K, \quad \mathbf{V} = \mathbf{X}\mathbf{W}_V \quad (6)$$

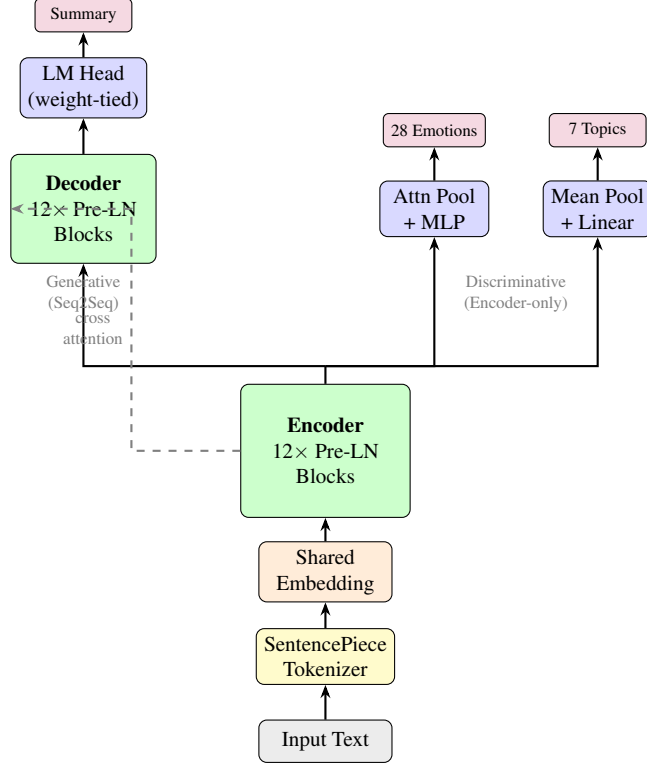


Figure 1: LEXIMIND system architecture. A shared encoder processes all input text. For summarization, encoder outputs are passed to a decoder with cross-attention and a weight-tied language modeling head. For classification tasks, encoder outputs are pooled (attention pooling for emotion, mean pooling for topic) and projected to class logits.

where $\mathbf{W}_Q, \mathbf{W}_K, \mathbf{W}_V \in \mathbb{R}^{d \times d_k}$. In multi-head attention, we split each projection into h heads of dimension $d_k = d/h$, compute attention independently per head, and concatenate:

$$\text{MultiHead}(\mathbf{X}) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) \mathbf{W}_O \quad (7)$$

Each head computes:

$$\text{head}_i = \text{softmax}\left(\mathbf{Q}_i \mathbf{K}_i^\top + \mathbf{B}_{\text{rel}}\right) \mathbf{V}_i \quad (8)$$

Note on scaling. The original Transformer [2] scales attention scores by $1/\sqrt{d_k}$ to prevent softmax saturation. T5 omits this scaling factor—the attention scores in Eq. (8) are *not* divided by $\sqrt{d_k}$. LEXIMIND follows T5’s convention via a configurable `scale_scores` parameter.

FlashAttention. When available, PyTorch 2.0’s `scaled_dot_product_attention` is used, which automatically selects the optimal kernel—FlashAttention v2 [11], memory-efficient attention, or a math fallback—based on hardware capabilities. This reduces memory from $O(n^2)$ to $O(n)$ in the fused case.

Figure 3 illustrates the attention computation flow.

3.2.1 Relative Position Bias

Instead of absolute positional embeddings added to token representations, T5 encodes position information as a bias added directly to attention scores. The relative position between positions i and j is mapped to a discrete bucket via logarithmic spacing:

$$B_{ij} = \text{Embedding}[\text{bucket}(i - j)] \quad (9)$$

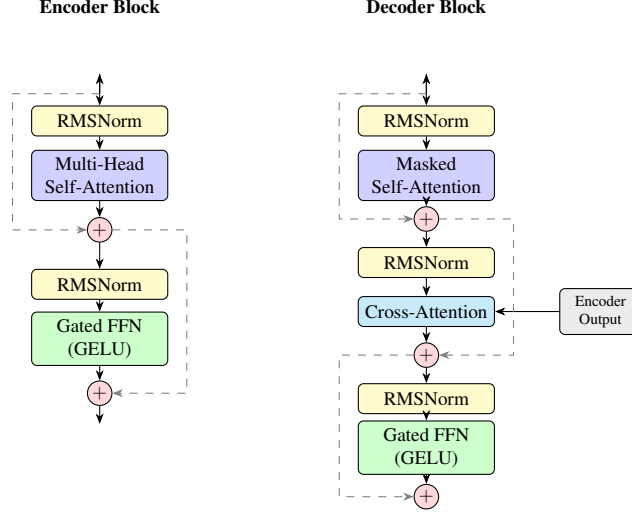


Figure 2: Pre-LN Transformer blocks. **Left:** Encoder block with self-attention and gated FFN. **Right:** Decoder block adding masked self-attention and cross-attention to encoder memory. RMSNorm is applied *before* each sublayer (Pre-LN). Dashed gray arrows show residual connections.

Half the buckets encode exact positions for nearby tokens (distances 0–7), while the remaining half use logarithmic spacing for distant tokens (up to 128). This allows the model to distinguish fine-grained local relationships while still representing long-range dependencies, all without adding parameters to the hidden states.

An important detail: in T5, only the first encoder layer and first decoder layer compute relative position biases. These biases are then reused (broadcast) across all subsequent layers, rather than computing independent biases per layer.

3.3 Gated Feed-Forward Network

Following T5, LEXIMIND uses a *gated* feed-forward network with GELU activation [14]:

$$\text{FFN}(\mathbf{x}) = (\text{GELU}(\mathbf{x}\mathbf{W}_{\text{gate}}) \odot \mathbf{x}\mathbf{W}_{\text{up}}) \mathbf{W}_{\text{down}} \quad (10)$$

where $\mathbf{W}_{\text{gate}}, \mathbf{W}_{\text{up}} \in \mathbb{R}^{d \times d_{\text{ff}}}$ and $\mathbf{W}_{\text{down}} \in \mathbb{R}^{d_{\text{ff}} \times d}$. The gating mechanism ($\text{GELU} \odot$) allows the network to selectively suppress information, providing more expressive transformations than the standard two-layer FFN. With $d = 768$ and $d_{\text{ff}} = 2048$, each gated FFN has three weight matrices instead of two, giving $3 \times 768 \times 2048 \approx 4.7\text{M}$ parameters per block.

3.4 RMSNorm

RMSNorm normalizes by the root mean square only, dropping the mean-centering operation of standard LayerNorm:

$$\text{RMSNorm}(\mathbf{x}) = \frac{\mathbf{x}}{\sqrt{\frac{1}{d} \sum_{i=1}^d x_i^2 + \epsilon}} \cdot \boldsymbol{\gamma} \quad (11)$$

where $\boldsymbol{\gamma} \in \mathbb{R}^d$ is a learned scale parameter and $\epsilon = 10^{-6}$. There is no bias term. This is computationally cheaper than LayerNorm (one fewer reduction) and empirically performs comparably [6].

3.5 Task-Specific Heads

The shared encoder produces contextualized representations $\mathbf{H} \in \mathbb{R}^{n \times d}$. Different tasks consume these representations through different heads.

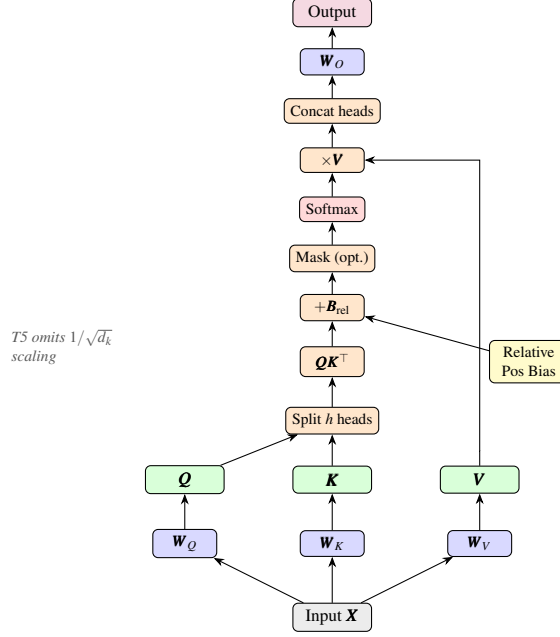


Figure 3: Multi-head attention with T5-style relative position bias. Attention scores are computed as $QK^\top + \mathbf{B}_{\text{rel}}$, where $\mathbf{B}_{\text{rel}}$ is a learned bias indexed by the relative distance between query and key positions. T5 does *not* scale by $1/\sqrt{d_k}$.

3.5.1 Summarization: Decoder + LM Head

Summarization uses the full encoder–decoder pipeline. Encoder outputs $\mathbf{H}$ are passed to the decoder via cross-attention, and the decoder autoregressively generates summary tokens. The language modeling head projects decoder hidden states to vocabulary logits, *with weights tied to the token embedding matrix*:

$$P(y_t \mid y_{<t}, \mathbf{x}) = \text{softmax}(\mathbf{h}_t^{\text{dec}} \mathbf{W}_{\text{embed}}^\top) \quad (12)$$

Training uses cross-entropy loss with label smoothing ($\epsilon = 0.1$) and `ignore_index=-100` for padding tokens. Inference uses greedy decoding with a maximum length of 128 tokens.

3.5.2 Emotion Classification: Attention Pooling + MLP

Emotion detection is a 28-label multi-label classification task—each text may express multiple emotions simultaneously. A naive approach would mean-pool the encoder outputs, but emotional content is typically concentrated in specific tokens or phrases, and mean pooling dilutes these signals across the full sequence.

Instead, we use *learned attention pooling*: a trainable weight vector $\mathbf{w} \in \mathbb{R}^d$ computes a per-position scalar score, and the output is a weighted sum of encoder hidden states:

$$\alpha_i = \frac{\exp(\mathbf{w}^\top \mathbf{h}_i)}{\sum_{j \in \text{valid}} \exp(\mathbf{w}^\top \mathbf{h}_j)} \quad (13)$$

$$\mathbf{h}_{\text{pool}} = \sum_{i \in \text{valid}} \alpha_i \mathbf{h}_i \quad (14)$$

where “valid” denotes non-padding positions. This allows the model to attend to emotionally salient tokens rather than treating all positions equally.

The pooled representation is then passed through a two-layer MLP:

$$\text{logits} = \mathbf{W}_2 \text{GELU}(\mathbf{W}_1 \mathbf{h}_{\text{pool}} + \mathbf{b}_1) + \mathbf{b}_2 \quad (15)$$

Table 1: LEXIMIND model specifications (base configuration).

Parameter	Value
Hidden dimension (d_{model})	768
FFN dimension (d_{ff})	2048
Attention heads (h)	12
Head dimension (d_k)	64
Encoder layers (N_{enc})	12
Decoder layers (N_{dec})	12
Vocabulary size	32,128
Max sequence length	512
Dropout	0.1
Activation	Gated-GELU
Normalization	RMSNorm (Pre-LN)
Position encoding	Relative bias
Total parameters	$\sim 272\text{M}$

where $\mathbf{W}_1 \in \mathbb{R}^{d/2 \times d}$ and $\mathbf{W}_2 \in \mathbb{R}^{28 \times d/2}$, giving a hidden dimension of 384. The MLP provides additional nonlinear capacity for the challenging 28-label task.

Training uses binary cross-entropy with logits loss (BCEWithLogitsLoss), enabling independent per-label prediction. At inference, per-label sigmoid probabilities are thresholded at 0.5 (or per-class tuned thresholds) to produce multi-label predictions.

3.5.3 Topic Classification: Mean Pooling + Linear

Topic classification assigns each document to exactly one of 7 categories: Arts, Business, Fiction, History, Philosophy, Science, and Technology. Because topical information is distributed more uniformly across the text (unlike emotion), simple mean pooling is effective:

$$\mathbf{h}_{\text{pool}} = \frac{\sum_{i=1}^n m_i \mathbf{h}_i}{\sum_{i=1}^n m_i} \quad (16)$$

where $m_i \in \{0, 1\}$ is the attention mask. A single linear layer projects to 7-way logits, trained with standard cross-entropy loss.

4 Weight Transfer from FLAN-T5

A key design goal of LEXIMIND is to combine the transparency of a custom implementation with the linguistic knowledge of a pre-trained model. The `factory.py` module handles this through a three-step pipeline: (1) load configuration, (2) instantiate custom modules, (3) map and transfer pre-trained weights.

4.1 Configuration

Architecture hyperparameters are defined in a YAML configuration file and loaded into a `ModelConfig` dataclass. Table 1 lists the values for the base configuration.

4.2 Weight Mapping

The pre-trained FLAN-T5-base model uses HuggingFace’s naming convention, which differs from our custom parameter names. The `factory` module maps between the two:

Table 2: Weight mapping from FLAN-T5 to LEXIMIND. The mapping is defined explicitly in `factory.py`, enabling transparent verification that all parameters are transferred.

FLAN-T5 Parameter	LEXIMIND Parameter
shared	encoder.embedding
*.SelfAttention.q	*.self_attn.W_Q
*.SelfAttention.k	*.self_attn.W_K
*.SelfAttention.v	*.self_attn.W_V
*.SelfAttention.o	*.self_attn.W_O
*.layer_norm	*.norm*
*.DenseReluDense.wi_0	*.ffn.linear_gate
*.DenseReluDense.wi_1	*.ffn.linear1
*.DenseReluDense.wo	*.ffn.linear2
lm_head	decoder.output_projection

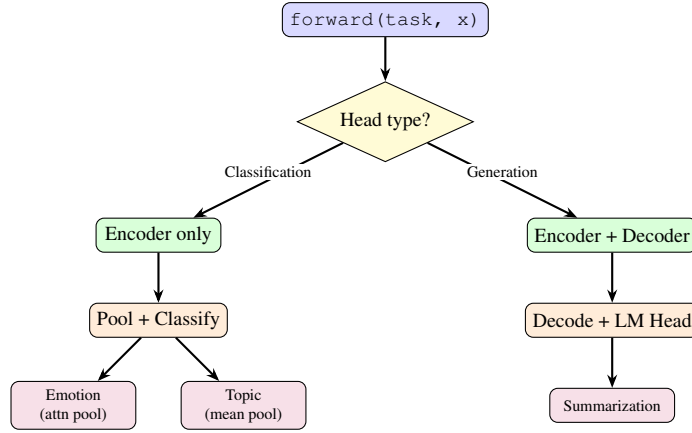


Figure 4: Task routing in `MultiTaskModel`. The head type determines whether encoder-only (classification) or full encoder-decoder (generation) processing is used.

4.3 Vocabulary Size Handling

T5 pads its vocabulary to multiples of 128 for GPU efficiency: the “real” vocabulary is 32,100 tokens, but the embedding matrix has 32,128 rows. During weight transfer, we copy only the tokens that exist in both vocabularies and initialize any extra rows with small random values.

4.4 Bias Initialization

T5’s attention layers have no bias terms, but our implementation includes optional biases for generality. When transferring weights, these bias terms are zero-initialized to ensure functional equivalence with the pre-trained model.

5 Multi-Task Learning

The `MultiTaskModel` class routes inputs to the appropriate processing path based on head type. Figure 4 illustrates the routing logic.

5.1 Temperature-Based Task Sampling

When training on multiple tasks with different dataset sizes (49K summarization, 43K emotion, 3.4K topic), uniform sampling would cause the model to see far more summarization and emotion examples

Table 3: Dataset statistics. All splits use consistent tokenization with a maximum length of 512 tokens.

Task	Source	Train	Val	Test
Summarization	Gutenberg + Goodreads	4K	–	–
	arXiv (body $\rightarrow$ abstract)	45K	–	–
	<i>Combined</i>	49,086	2,727	2,727
Topic (7-class)	20News + Gut. + arXiv	3,402	189	189
Emotion (28-label)	GoEmotions	43,410	5,426	5,427
Total		95,898	8,342	8,343

per epoch than topic examples. We use temperature-based sampling [15] to balance task exposure:

$$p_t = \frac{|D_t|^{1/\alpha}}{\sum_{t'} |D_{t'}|^{1/\alpha}} \quad (17)$$

where $|D_t|$ is the size of dataset t and α is the temperature parameter. With $\alpha = 0.5$ (our setting), smaller datasets are upsampled relative to their natural frequency, preventing them from being overwhelmed by larger tasks.

5.2 Task Loss Weighting

The total loss combines task-specific losses with explicit weights:

$$\mathcal{L}_{\text{total}} = \lambda_{\text{summ}} \mathcal{L}_{\text{summ}} + \lambda_{\text{emo}} \mathcal{L}_{\text{emo}} + \lambda_{\text{topic}} \mathcal{L}_{\text{topic}} \quad (18)$$

We set $\lambda_{\text{summ}} = \lambda_{\text{emo}} = 1.0$ and $\lambda_{\text{topic}} = 0.3$. The reduced topic weight prevents the small topic dataset from dominating shared gradients—without this, topic classification converges to near-perfect training accuracy within 2–3 epochs while degrading performance on other tasks.

6 Training Pipeline

6.1 Datasets

Table 3 summarizes the training data for each task.

Summarization. Literary pairs match Project Gutenberg full texts with Goodreads back-cover descriptions via title/author matching. Academic pairs use arXiv paper bodies with their abstracts. The academic subset is substantially larger ($\sim 45\text{K}$ vs. $\sim 4\text{K}$ literary), creating an 11:1 domain imbalance that favors academic summarization quality (Section 7).

Topic classification. Labels are derived from source metadata (arXiv categories, Gutenberg subjects, 20 Newsgroups categories) and mapped to a 7-class taxonomy. No manual annotation was performed, introducing potential noise from metadata inaccuracies.

Emotion detection. GoEmotions [12] provides 28 fine-grained emotion labels derived from Reddit comments. The 28 labels are: admiration, amusement, anger, annoyance, approval, caring, confusion, curiosity, desire, disappointment, disapproval, disgust, embarrassment, excitement, fear, gratitude, grief, joy, love, nervousness, neutral, optimism, pride, realization, relief, remorse, sadness, and surprise.

Cross-task deduplication. Because the topic dataset draws from the same sources as summarization (arXiv, Gutenberg), we perform cross-task document deduplication using MD5 fingerprints of normalized text prefixes to prevent data leakage.

6.2 Tokenization

LEXIMIND wraps HuggingFace’s `AutoTokenizer` with a simplified façade handling T5-specific conventions. T5 uses SentencePiece [9] unigram tokenization with a 32,128-token vocabulary. Notable

Table 4: Training hyperparameters.

Hyperparameter	Value
Optimizer	Fused AdamW
Learning rate	3×10^{-5}
Weight decay	0.01
Betas	(0.9, 0.98)
Batch size	10 (effective 40 with $4 \times$ accum.)
LR schedule	Cosine with linear warmup
Warmup steps	300
Min LR ratio	$0.1 \times$ peak
Max epochs	8
Early stopping patience	3
Precision	BFloat16 (AMP)
Gradient clipping	Max norm 1.0
Gradient checkpointing	Enabled
<code>torch.compile</code>	Encoder and decoder
Label smoothing	0.1 (summarization)
Task weights	Summ. 1.0, Emo. 1.0, Topic 0.3
Task sampling	Temperature ($\alpha = 0.5$)
Frozen encoder layers	Bottom 4 of 12

conventions: `pad_token_id=0`, `eos_token_id=1`, no explicit BOS token (the pad token serves as the decoder start token).

For seq2seq training, decoder inputs are created by right-shifting the target labels:

Listing 1: Decoder input preparation

```
decoder_inputs[:, 0] = bos_token_id # = pad_token_id for T5
decoder_inputs[:, 1:] = labels[:, :-1]
```

6.3 Training Configuration

All training was conducted on a single NVIDIA RTX 4070 (12 GB VRAM). Table 4 lists the hyperparameters.

Mixed-precision training. We use PyTorch AMP with BFloat16; on Ampere+ GPUs, bfloat16 provides better numerical stability than float16 (no loss scaling needed) while maintaining equivalent speed benefits. TF32 is enabled for matrix multiplications.

Learning rate schedule. A linear warmup over 300 steps (~ 0.5 epochs) prevents early training instability. After warmup, the learning rate follows a cosine decay to 10% of its peak value:

$$\eta(t) = \eta_{\min} + \frac{1}{2}(\eta_{\max} - \eta_{\min})(1 + \cos(\pi \cdot \text{progress})) \quad (19)$$

where $\text{progress} = (t - t_{\text{warmup}}) / (T - t_{\text{warmup}})$ and $\eta_{\min} = 0.1 \cdot \eta_{\max}$.

Encoder freezing. The bottom 4 encoder layers are frozen during fine-tuning. These layers capture general linguistic features (morphology, syntax) that transfer well across tasks; freezing them preserves these representations while reducing memory and compute, and allowing upper layers to specialize for our domain.

CUDA optimizations. We enable `cudnn.benchmark`, TF32 for matmul, FlashAttention via SDPA, and memory-efficient attention kernels.

7 Experimental Results

We evaluate on held-out validation sets. All summarization metrics report mean and 95% bootstrap confidence intervals. Training completed all 8 epochs (approximately 9 hours) without triggering early

Table 5: Summarization results on the validation set ($n = 2,727$).

Metric	Score
ROUGE-1	0.310 [0.306, 0.313]
ROUGE-2	0.091 [0.088, 0.094]
ROUGE-L	0.185 [0.182, 0.188]
BLEU-4	0.024
BERTScore Precision	0.843
BERTScore Recall	0.818
BERTScore F1	0.830

Table 6: Per-domain summarization performance.

Domain	N	ROUGE-1	ROUGE-L	BLEU-4
Academic	2,493	0.319	0.189	0.026
Literary	234	0.206	0.137	0.008

stopping.

7.1 Summarization

The high BERTScore F1 (0.83) combined with moderate ROUGE-1 (0.31) is characteristic of abstractive summarization: the model generates semantically faithful paraphrases rather than extracting text verbatim. ROUGE, which measures n-gram overlap, undervalues such paraphrasing; BERTScore’s contextual embeddings [13] better capture semantic similarity.

Per-domain analysis. The 11:1 training imbalance produces a clear quality gap: academic summaries achieve ROUGE-1 of 0.319 while literary summaries reach only 0.206. This is expected—the model sees far more academic text during training and the formal structure of academic abstracts provides clearer extractive cues.

7.2 Topic Classification

Table 8 shows per-class performance. Fiction and Business achieve near-perfect F1 (≥ 0.97), while Science shows the most confusion ($F1 = 0.67$)—error analysis reveals frequent misclassification as Technology, which is semantically plausible given that scientific papers often describe technical methods.

7.3 Emotion Detection

Emotion detection is evaluated with three complementary F1 metrics, as each captures different aspects of multi-label performance:

At the default threshold, 15 of 28 emotion classes have zero F1. These tend to be rare classes (< 100 support) or semantically subtle emotions (anger, annoyance, approval) that overlap with other labels. Per-class threshold tuning—sweeping $\tau \in \{0.1, 0.15, \dots, 0.9\}$ per class—recovers non-zero performance for most classes, doubling macro F1 from 0.143 to 0.294.

The strongest classes are gratitude ($F1 = 0.89$), amusement (0.75), love (0.74), and admiration (0.65)—these have clear lexical signals and reasonable support (> 250 samples). The model struggles most with rare, nuanced emotions like grief (13 samples), embarrassment (35), and pride (15).

Table 7: Topic classification results on the validation set ($n = 189$).

Metric	Score
Accuracy	85.7% [80.4%, 91.0%]
Macro F1	0.854

Table 8: Per-class topic classification performance.

Topic	Precision	Recall	F1
Arts	0.93	0.79	0.86
Business	0.97	1.00	0.98
Fiction	0.95	1.00	0.97
History	0.85	0.76	0.80
Philosophy	0.79	0.82	0.81
Science	0.57	0.80	0.67
Technology	0.89	0.89	0.89
<i>Macro Avg</i>	0.85	0.87	0.85

7.4 Training Dynamics

Training completed 8 full epochs (no early stopping triggered). Combined validation loss decreased monotonically from 4.297 (epoch 1) to 3.925 (epoch 8). Validation emotion sample F1 rose steadily from 0.197 to 0.459, while topic accuracy plateaued after epoch 5 at $\sim 86\%$.

The different learning trajectories illustrate a key MTL phenomenon: the small topic dataset saturates quickly (97% training accuracy by epoch 3), while emotion detection—with its larger dataset and harder 28-class problem—continues improving throughout. The reduced topic weight ($\lambda = 0.3$) and temperature sampling prevent the saturated topic task from degrading the shared encoder.

7.5 Baseline Comparison: BERT

To disentangle the effects of architecture from multi-task learning, we compare LEXIMIND against BERT-base-uncased [1] (110M parameters) in three configurations: single-task topic, single-task emotion, and joint multitask. The BERT baselines use identical training hyperparameters (learning rate, batch size, gradient accumulation, epochs) and the same data splits, with task-specific classification heads matching LEXIMIND’s design: attention pooling + 2-layer MLP for emotion, mean pooling + linear projection for topic.

Table 10 reveals several instructive patterns:

BERT dominates classification. The encoder-only architecture substantially outperforms LEXIMIND on both classification tasks in single-task settings (+1.6% topic accuracy, +72% relative emotion sample F1). This is expected: BERT’s bidirectional encoder produces richer contextual representations for classification than using only the encoder half of an encoder–decoder model, and BERT can devote all parameters to the classification objective.

MTL hurts BERT on topic. BERT’s multitask configuration drops topic accuracy by 7.4 percentage points compared to its single-task result (79.9% vs. 87.3%), while emotion performance remains essentially unchanged. This mirrors the negative transfer LEXIMIND avoids through temperature sampling and attention pooling—the same interventions are absent in the BERT multitask setup.

Table 9: Emotion detection results on the validation set ($n = 5,426$). Default uses a fixed threshold of 0.3; tuned uses per-class thresholds optimized on the validation set.

Metric	Default ($\tau = 0.3$)	Tuned
Sample-avg F1	0.352 [0.340, 0.366]	0.503
Macro F1	0.143	0.294
Micro F1	0.443	0.486

Table 10: Comparison of LEXIMIND (encoder–decoder MTL) with BERT-base baselines. Single-task models are trained on one task only; multitask trains on both classification tasks jointly. Bold indicates best per metric; † denotes tuned per-class thresholds.

Metric	LEXIMIND (MTL)	BERT Single Topic	BERT Single Emotion	BERT MTL
<i>Topic Classification</i>				
Accuracy	85.7%	87.3%	—	79.9%
Macro F1	0.854	0.878	—	0.776
<i>Emotion Detection</i>				
Sample-avg F1	0.352	—	0.608	0.607
Macro F1	0.143	—	0.469	0.487
Micro F1	0.443	—	0.607	0.600
Macro F1 [†]	0.294	—	0.512	0.510
<i>Summarization</i>				
BERTScore F1	0.830	—	—	—

Architecture enables generation. The BERT baselines cannot perform abstractive summarization—this capability requires the decoder component unique to LEXIMIND’s encoder–decoder design. The classification performance gap is the cost of architectural flexibility: LEXIMIND uses a single model for all three tasks, including a generative task that encoder-only models cannot address.

8 Discussion

8.1 Why Attention Pooling Matters

The choice of pooling mechanism for classification heads has outsized impact. Mean pooling works well for topic classification because topical signals are distributed across the text—a paper about machine learning will contain relevant vocabulary throughout. But emotional content is localized: a single phrase (“I’m devastated”) may carry all the emotional signal, diluted by hundreds of neutral tokens under mean pooling.

Learned attention pooling lets the model focus on these salient positions. Switching from mean pooling to attention pooling for the emotion head—with no other changes—improved sample-averaged F1 from 0.199 to 0.352 (+77%), completely eliminating the negative transfer observed in the naive MTL configuration.

8.2 The BERTScore vs. ROUGE Gap

The large gap between BERTScore F1 (0.83) and ROUGE-1 (0.31) deserves explanation. ROUGE measures *lexical* overlap—it counts shared n-grams between generated and reference texts. BERTScore computes cosine similarity between contextual embeddings, capturing *semantic* similarity regardless of exact wording.

For abstractive summarization, this gap is expected and even desirable: it means the model is paraphrasing (generating semantically equivalent text with different words) rather than copying. Our summarization head generates back-cover style descriptions rather than extractive summaries, so high BERTScore with moderate ROUGE indicates the model is behaving as intended.

8.3 Domain Imbalance in Summarization

The 11:1 academic-to-literary training ratio produces a clear quality asymmetry: academic ROUGE-1 of 0.319 vs. literary ROUGE-1 of 0.206. Beyond raw volume, academic text provides stronger extractive cues—paper abstracts follow predictable structures (“We propose...”, “Experiments show...”) that transfer well to compression. Literary descriptions are more open-ended, and the model’s exposure to contemporary fiction is limited by Project Gutenberg’s focus on pre-1928 public domain works.

8.4 Limitations

- **Classification–generation tradeoff:** The BERT baseline comparison (Table 10) confirms that the encoder–decoder architecture underperforms encoder-only models on pure classification. This is the cost of supporting generative tasks within a single model.
- **Emotion–domain mismatch:** GoEmotions is sourced from Reddit comments, which differ sharply in register from the literary and academic content LEXIMIND targets. The 28-class multi-label setting compounds this difficulty.
- **Topic data scarcity:** With only 3,402 training samples across 7 classes, some categories (notably Science, $F1 = 0.67$) lack sufficient examples. The Science–Technology boundary is inherently ambiguous.
- **Literary generation quality:** The domain imbalance limits literary summarization. Balancing the training mix or augmenting with contemporary book data would likely help.
- **Greedy decoding:** We use greedy decoding for simplicity; beam search or nucleus sampling could improve generation diversity and quality.

8.5 Future Directions

- **LoRA [10]:** Parameter-efficient fine-tuning would reduce memory requirements, enabling experimentation with FLAN-T5-large (780M) or FLAN-T5-xl (3B).
- **Domain-specific emotion data:** Training on literary emotion annotations rather than Reddit comments would better capture the emotional register of books and papers.
- **Threshold learning:** Replacing the fixed sigmoid threshold with a learned decision boundary per class could improve multi-label performance without post-hoc tuning.
- **Data augmentation:** Back-translation or LLM-based paraphrasing for the small topic dataset could improve classification robustness.

9 Conclusion

We presented LEXIMIND, a multi-task NLP system that combines a custom Transformer reimplementation with FLAN-T5 pre-trained weights. The hybrid approach provides full architectural transparency while leveraging transfer learning, achieving:

- **Summarization:** BERTScore F1 of 0.83, demonstrating strong semantic fidelity for abstractive literary and academic summaries.
- **Topic Classification:** 85.7% accuracy (macro $F1 = 0.854$) across 7 categories.
- **Emotion Detection:** Sample-averaged F1 of 0.352 (tuned macro $F1 = 0.294$) on 28 emotion labels.

Two interventions proved critical for balanced multi-task learning: learned attention pooling for the emotion head, and temperature-based task sampling. Together, these eliminated negative transfer and improved emotion F1 by 77% over the naive MTL baseline.

A controlled BERT-base comparison (Table 10) clarifies the architecture tradeoff: BERT achieves stronger classification performance (87.3% topic accuracy, 0.608 emotion sample F1) but cannot perform abstractive summarization. The encoder–decoder design sacrifices classification accuracy in exchange for generative capabilities—a deliberate tradeoff that makes LEXIMIND suitable for applications requiring both understanding and generation.

The complete system trains in approximately 9 hours on a consumer GPU (NVIDIA RTX 4070, 12 GB), demonstrating that sophisticated multi-task models remain accessible without datacenter-scale resources. All code, trained weights, and datasets are publicly available.

References

- [1] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova. BERT: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of NAACL-HLT*, pages 4171–4186, 2019.
- [2] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 30, pages 5998–6008, 2017.
- [3] C. Raffel, N. Shazeer, A. Roberts, K. Lee, S. Narang, M. Matena, Y. Zhou, W. Li, and P. J. Liu. Exploring the limits of transfer learning with a unified text-to-text transformer. *Journal of Machine Learning Research*, 21(140):1–67, 2020.
- [4] H. W. Chung, L. Hou, S. Longpre, B. Zoph, Y. Tay, W. Fedus, et al. Scaling instruction-finetuned language models. *arXiv preprint arXiv:2210.11416*, 2022.
- [5] R. Xiong, Y. Yang, J. He, K. Zheng, S. Zheng, C. Xing, H. Zhang, Y. Lan, L. Wang, and T. Liu. On layer normalization in the transformer architecture. In *International Conference on Machine Learning (ICML)*, pages 10524–10533, 2020.
- [6] B. Zhang and R. Sennrich. Root mean square layer normalization. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 32, pages 12360–12371, 2019.
- [7] J. L. Ba, J. R. Kiros, and G. E. Hinton. Layer normalization. *arXiv preprint arXiv:1607.06450*, 2016.
- [8] R. Caruana. Multitask learning. *Machine Learning*, 28(1):41–75, 1997.
- [9] T. Kudo and J. Richardson. SentencePiece: A simple and language independent subword tokenizer and detokenizer for neural text processing. In *Proceedings of EMNLP: System Demonstrations*, pages 66–71, 2018.
- [10] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations (ICLR)*, 2022.
- [11] T. Dao, D. Fu, S. Ermon, A. Rudra, and C. Ré. FlashAttention: Fast and memory-efficient exact attention with IO-awareness. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 35, pages 16344–16359, 2022.
- [12] D. Demszky, D. Movshovitz-Attias, J. Ko, A. Cowen, G. Nemade, and S. Ravi. GoEmotions: A dataset of fine-grained emotions. In *Proceedings of ACL*, pages 4040–4054, 2020.
- [13] T. Zhang, V. Kishore, F. Wu, K. Q. Weinberger, and Y. Artzi. BERTScore: Evaluating text generation with BERT. In *International Conference on Learning Representations (ICLR)*, 2020.

- [14] D. Hendrycks and K. Gimpel. Gaussian Error Linear Units (GELUs). *arXiv preprint arXiv:1606.08415*, 2016.
- [15] N. Arivazhagan, A. Bapna, O. Firat, D. Lepikhin, M. Johnson, M. Krikun, et al. Massively multilingual neural machine translation in the wild: Findings and challenges. *arXiv preprint arXiv:1907.05019*, 2019.